

Designed and engineered, artfully — silicon to software.

BSc Computer Science, Manchester (2023–2026) · Incoming MSc Computer & Embedded Systems Engineering, TU Delft (Sept 2026)

01	Quant Trading System (Automaton-QTS)	FINANCE
02	Cellular-Automaton SoC Accelerator	FINANCE
03	Aero-Lab — Quadrotor Flight Sim	DRONE
04	RRHC vs MPCC: A Reactive Racing Controller That Beats Online Optimisation	MOTORSPORT
05	Stump — 16-bit RISC CPU closed on Spartan-6 silicon	FINANCE
06	Multicore Computing Labs	FINANCE
07	ONI — Local-LLM Agent Harness	FINANCE
08	DAWgit — Git for Ableton Projects	DRONE
09	FSAI Driverless Racing Simulator	MOTORSPORT
10	CommonRoad C++ Port	MOTORSPORT
11	Slit Light Interference Simulator	DRONE
12	CarrLane — Engineering Part Catalog Chatbot	FINANCE

Quant Trading System (Automaton-QTS)

A multi-signal trading engine, an agent-based market simulator, and a relation-typed contagion-propagation graph that I proved is dead in equities and alive in crypto.

Python PyTorch torchdiffeq NautilusTrader hftbacktest scipy

CODE 45.7k LOC Python	TESTS 1,314 test fns, 81.9% cov	CRYPTO CONTAGION (RESEARCH-STAGE, N=3) +9.2%/event mkt-neutral, Sharpe 1.49 (n=3, small- sample feasibility study)	TERRA LINKED-PEER SIGNAL (RESEARCH-STAGE, N=3) p=0.0009 (Mann- Whitney, 72h)
---	---	--	--

ROLE

Sole author. Built the signal/risk/execution core, the human-gated LLM oversight trail, an agent-based market simulator for synthetic data, and the relation-typed propagation-graph research line (equity negative + crypto positive).

REPO / <https://github.com/RandevRanjit/Automaton-QTS>

Cellular-Automaton SoC Accelerator

A from-scratch SystemVerilog drawing unit that renders Wolfram cellular automata — programmable rule as a pure lookup, LFSR-seeded, taken through Vivado to silicon and demoed live on a monitor.

SystemVerilog Vivado Artix-7 RISC-V assembly Questa

ACCELERATOR (BLOCK) 4,433 LUT / 1,467 FF / 0 BRAM	WHOLE SOC FILL 43.81% LUT • 96.0% BRAM	RULE SPACE 256 rules, 1 register	FUNCTIONAL COVERAGE 100% stmt/ branch/cond/ FSM
---	--	--	---

ROLE

Designed unit_automaton from scratch (programmable rule-as-lookup, 32-bit LFSR seed, 3-state compute/write FSM, byte-lane framestore writes with req/ack backpressure), wrote a coverage-driven testbench to 100% functional coverage, integrated it into the drawing engine replacing a dummy slot, drove it from a RISC-V MMIO program, and closed it through Vivado synthesis + implementation on Artix-7 with a live monitor demo.

PRIVATE · CASE STUDY (NO CODE LINK)

05 / HARDWARE

Stump — 16-bit RISC CPU closed on Spartan-6 silicon

A multi-cycle 16-bit RISC processor built bottom-up from RTL primitives in Verilog, then synthesised, placed-and-routed, and timing-closed on a real Spartan-6 at 98% slice occupancy — the first design I carried through the full FPGA back-end flow.

Verilog-2001 Xilinx ISE Spartan-6 (xc6slx4) ModelSim Stump assembly

SLICE LUTS 2,068 / 2,400 (86%)	OCCUPIED SLICES 591 / 600 (98%)	SLICE REGISTERS 1,052 / 4,800 (21%)	TIMING SCORE 0 – setup 0, hold 0, all signals fully routed
--	---	---	--

ROLE

Authored the datapath (ALU, barrel shifter, sign-extender, 8-register file with R0=0 / R7=PC, operand muxes), the 3-state control FSM (FETCH/EXECUTE/MEMORY) and the control-decode logic, and implemented the LDST and BCC instruction paths. Top-level board wrapper and 1.5k-line memory model were provided. Wrote test programs in Stump assembly including a Battleship game driving a memory-mapped 8×8 LED matrix.

PRIVATE · CASE STUDY (NO CODE LINK)

06 / SYSTEMS

Multicore Computing Labs

Three parallelism labs on a 72-core 2-socket NUMA machine — a 132× temporal-blocking Jacobi stencil, a 30.7× NUMA-aware vecadd, and dining philosophers scaling near-linearly to 1.04M meals/s.

C C++20 pthreads OpenMP std::barrier NUMA

TEMPORAL-BLOCKING STENCIL 132.5× @ 72T (124.7× 10-95%) SYNC REDUCTION (STENCIL) 1 barrier / 128 iters vs 2 / iter	OPENMP STENCIL 128.6× @ 72T (116.7× 10-95%)	NUMA VECADD SPEEDUP 30.7× @ 72T on 256M doubles	FINE-SPINLOCK PHILOSOPHERS 1,039K meals/ s @ 64T (mean of 5)
---	--	--	---

ROLE

Sole author (s94810rr), three labs. Implemented NUMA first-touch pthreads vecadd; four-variant dining philosophers (coarse/fine × mutex/spinlock) with lock-ordering deadlock prevention; and a temporal-blocking 1-D Poisson stencil in both std::thread+std::barrier and OpenMP. Benchmarked on mcore72 (2× Xeon Platinum 8452Y, 72 cores, 2 NUMA nodes).

PRIVATE · CASE STUDY (NO CODE LINK)

07 / SYSTEMS

ONI — Local-LLM Agent Harness

79k-LOC single-crate Rust harness driving a local llama.cpp / MLX server through a 33-tool fenced-code protocol, a graph working-memory, and harness-managed compaction — with 2,151 tests and µs/ns Criterion latency contracts.

Rust

tokio

request

criterion

llama.cpp

MLX

CODEBASE 79,240 LOC Rust / 143 files / 1 crate LOOP-DETECTOR BUDGET < 50 ns / 1 KB SSE chunk	TESTS 2,151 unit/ async tests BACKENDS 3 (llama.cpp, MLX, oMLX)	PARSER BUDGET < 100 µs / 16-fixture corpus	TOOL DISPATCH BUDGET < 5 ms / call
---	---	--	--

ROLE

Sole author. Designed and built the whole harness: the OMTF v0.4 fenced-code tool protocol and its 33-tool parser, the backend seam abstracting three inference servers, the graph working-memory with BFS gather and harness-managed compaction, the LIT/SIT stream-loop detectors, and the process-group SIGKILL sandbox. Every behaviour-changing feature is greenlit, rollback-tracked, and validated by scored A/B runs.

PRIVATE REPOSITORY

CarrLane — Engineering Part Catalog Chatbot

Three-tier function-calling assistant that turns natural-language part queries into typed API calls over a scraped engineering catalog — built during an industry internship.

Python

FastAPI

SQLModel

Node/Express

React

Chakra UI

CATALOG (PROTOTYPE) 314 parts / 11 product types DATES Jun-Aug 2025, Chennai	COMPANY CATALOG 10,000+ parts	LLM TOOLS 9 typed function schemas	API TESTS 8 async endpoint tests
--	---	--	--

ROLE

AI / Software Engineering Intern. Built the catalog ingestion, the typed REST API over it, and the function-calling orchestration layer that maps natural-language queries to catalog lookups.

PRIVATE REPOSITORY

FIELD / DRONE / MUSIC

Aero-Lab — Quadrotor Flight Sim

Hand-rolled 6-DOF quadrotor dynamics plus four racing controllers — PID, MPCC (acados SQP-RTI), a reactive RRHC, and a PPO policy — with PID/MPCC/RRHC benchmarked head-to-head on a 12-course fairness harness.

Python

NumPy

SciPy

acados

CasADi

Gymnasium

TESTS 357 passing (4 skip, 1 xfail)	CONTROLLERS 4 racing + 2 cinematic	RRHC VS MPCC fastest on 10/12 courses, 162x cheaper/tick	CODE ~8.5k src + 6.9k test LOC
---	--	--	--

ROLE

Sole author. Hand-rolled the 6-DOF rigid-body dynamics (RK4, motor lag, gyroscopic torque, drag), the analytic differential-flatness inner loop shared by every controller, an acados SQP-RTI MPCC, a CTBR PPO env + policy, and a fairness contract (pre-flight + watchdog + post-hoc audit) that enforces identical plant, course, and control rate across all controllers.

PRIVATE REPOSITORY

DAWgit — Git for Ableton Projects

A macOS daemon that gives Ableton Live real version control — diffing the music, not the bytes. Semantic .als diff, per-file branching that never touches HEAD, SHA-256 audio LFS, over a Unix-socket IPC bus.

Rust tokio libgit2 (git2-rs) roxmltree flate2 SHA-256

.ALS → TYPED AST gzip → roxmltree DOM walk → ProjectAst, 1,306 LOC parser TESTS 52 passing (18 parser/ differ, 15 git, 19 IPC)	SEMANTIC DIFF 21 delta variants (track/clip/ device/ automation/ global), ID-keyed set diff	PER-FILE BRANCH COMMIT new tree on refs/heads/ , HEAD/ index/workdir untouched	AUDIO LFS SHA-256 content-addressed cache + git-lfs v1 pointers, dedup on existence
--	---	---	---

ROLE

Sole author. Built the three-binary system end to end — Rust daemon (FS watcher, async/sync IPC boundary, git object layer, LFS, project discovery, launchd integration), the 1,306-LOC Ableton gzip-XML parser, the semantic differ, the clap CLI, and the SwiftUI menubar app that talks to the daemon over the socket.

PRIVATE REPOSITORY

Slit Light Interference Simulator

A 3D single-slit Fraunhofer diffraction lab — the sinc² intensity envelope solved per-fragment on the GPU, wrapped in a Three.js scene with draggable apparatus, live wavelength/slit/distance controls and a real-time measurement readout.

JavaScript Three.js GLSL Tweakpane Webpack

DIFFRACTION MODEL sinc² Fraunhofer envelope, per-fragment	SHADER PROGRAMS 3 (grid / interference / loading), 6 GLSL files	ENGINE ~1.65k LOC single-file Three.js app	WAVELENGTH→COLOUR 380–780 nm CIE piecewise → RGB
--	---	---	---

ROLE

Sole author. Wrote the interference vertex/fragment shader pair (sinc² envelope, SI-unit parametrisation), the wavelength→RGB conversion, the full Three.js scene graph, the draggable transform-controlled apparatus, the live measurement panel, and the particle-haze system with its sorted exponential-search culling.

REPO / <https://github.com/RandevRanjit/Slit-Light-Interference-Sim>

FIELD / MOTORSPORT

04 / CONTROL

RRHC vs MPCC: A Reactive Racing Controller That Beats Online Optimisation

Final-year dissertation — a model-free heuristic racing controller, benchmarked against an IPOPT-solved MPCC under an enforced fairness contract.

Python NumPy SciPy CasADi IPOPT Optuna

LAP TIME VS MPCC 12.2% faster (164.2s vs 187.1s, 3 laps)	PER-STEP COMPUTE 0.13 ms vs 19.88 ms (153x)	DEADLINE OVERRUNS (ZOH) 0 vs 1,870 over baseline trial	CROSS-PHASE SIGNIFICANCE Wilcoxon W=253, p=1.9e-5
--	--	---	--

ROLE

Sole author (dissertation). Designed and built the RRHC controller, the MPCC/IPOPT baseline, the 3-DOF Fiala-tyre plant, the fairness-contract harness, and the five-phase statistical evaluation.

PRIVATE REPOSITORY

FSAI Driverless Racing Simulator

I architected and authored the simulation engine (~29k of 34k LOC C++) — RK4 with adaptive sub-stepping over three CommonRoad vehicle models (7/9/29 state), an OpenGL stereo camera with PBO readback, and ns-budget timing across a software CAN bus. Built on an 11-person Formula Student AI team; I wrote the physics, IO, CAN stack, and common infrastructure. Perception pipeline by teammates.

C++17

CMake

Eigen

OpenGL

SDL2

ONNXRuntime

CODEBASE 34,148 LOC C++ + (11-person team; ~29k authored by me)	VEHICLE DYNAMICS RK4 + adaptive sub-stepping, 3 models (7/9/29-state)	STEREO CAMERA FBO render → PBO glReadPixels readback → ring buffer	REAL-TIME BUDGETS ns-resolution timers, 4 subsystems, swappable clock
--	--	---	--

ROLE

Architect and sole author of the simulation engine (~29k of 34k LOC). Designed and wrote the velox physics core (RK4 + adaptive sub-stepping, all three CommonRoad vehicle models), the sim loop, the OpenGL FBO stereo camera with PBO readback, the AI-to-VCU CAN interface and UDP S-VCU link, the ns-budget timing + swappable-clock infrastructure, and all common/IO/control libraries. 11-person team project; the ONNX/SIFT/Kalman perception pipeline was teammates' work — it plugs into the camera and pose feeds I built.

REPO / <https://github.com/RandevRanjit/FSAI-Simulation-C>

CommonRoad C++ Port

~10.6k LOC C++20 port of the CommonRoad vehicle models (ST / STD / MB) into a static library, derivative outputs matched bit-for-bit against the Python reference to 1e-13, plus a ~4.5k LOC TypeScript SDK whose JS backend is parity-checked field-by-field against the native one.

C++20

CMake

TypeScript

YAML

SDL2/ImGui

CODEBASE ~10.6k LOC C++ + ~4.5k LOC TS	SELECTABLE MODELS ST / STD / MB (29-state)	DERIVATIVE PARITY VS PYTHON ST/STD to 1e-13, MB to 1e-7	TEST SUITE 19 test files, 19 CTest targets
---	---	--	---

ROLE

Sole author. Ported the CommonRoad ST/STD/MB vehicle models from the academic Python reference to a C++20 static library ('velox'), built the 'SimulationDaemon' API with a 'ModelTiming' sub-step scheduler, Pacejka tyre model, EV powertrain controller, staged low-speed/loss-of-control safety, and a TypeScript web SDK with a JS backend parity-checked against the native one.

REPO / <https://github.com/RandevRanjit/CommonRoad-CXX-Port>